

3 Gebruikte hardware

In dit deel beschrijven we wat we zullen gebruiken van hardware.

3.1 STM32F091RC

De keuze van de microcontroller is bij de aanvang van een embedded project heel belangrijk. Zaken waar men zeker rekening moet mee houden zijn:

- Wat mag de kostprijs zijn?
- Hoe vlot kunnen we de microcontrollers aankopen?
- Hoeveel bordruimte (grootte) is er beschikbaar op de print?
- Hoeveel berekeningen per seconde willen we hebben?
- Op welke spanning willen we werken?
- Welk stroomverbruik is toegelaten?
- Hoeveel IO's hebben we nodig?
- Welke communicatiekanalen willen we gebruiken (UART, SPI, CAN, ...)?
- In welke programmeertaal willen we programmeren?
- ...

Met bovenstaande in het achterhoofd is het onmogelijk om de perfecte microcontroller te vinden, maar hebben we de STM32F091RC-microcontroller gekozen om volgende redenen:

- 32-bit microcontroller (grote rekenkracht)
- Tot 48 MHz klokfrequentie (tamelijk grote rekenkracht)
- Alle basis communicatiekanalen zijn aanwezig (UART, SPI, I²C, CAN, ...)
- Prijs
- Het is een ARM microcontroller en wordt wellicht 'de' standaard de komende jaren.
- Snel beschikbaar via ontwikkelborden
- De ontwikkelborden zijn uitbreidbaar via Arduino compatibele stacking headers en de microcontroller is programmeerbaar via USB-interface
- ...

De specifieke specificaties van de microcontroller zijn terug te vinden in Figuur 4.

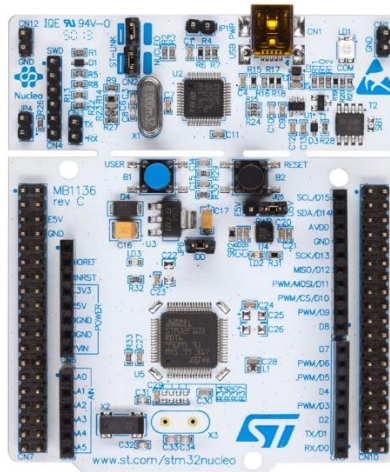
De microcontroller is beschikbaar in LQFP64, dus het dient om op een printplaat (PCB) te worden geplaatst en is daardoor minder geschikt voor directe plaatsing op bijvoorbeeld een breadboard of gaatjesprint. Dit maakt het lastig om snel te testen en prototypes te bouwen.

Om dit probleem op te lossen, heeft STMicroelectronics een reeks **ontwikkelborden**, de **Nucleo-borden**, ontworpen.

3.2 NUCLEO-F091RC

Een Nucleo-bord maakt het eenvoudig om verbindingen te kunnen leggen met jumperkabels, (Arduino) shields te gebruiken, code te programmeren via USB-interface en de microcontroller te debuggen. Het Nucleo-bord bevat daarnaast een drukknop, een resetknop en een LED.

In deze cursus zullen we gebruikmaken van het **NUCLEO-F091RC-bord** waarbij het typenummer uiteraard verwijst naar de gebruikte microcontroller.



Figuur 9: Nucleo-bord

3.2.1 Schema

Het schema van het Nucleo-bord is gemeenschappelijk met andere Nucleo-borden waarop een 64-pin ARM op gesoldeerd is. Dit geeft onmiddellijk ook aan dat STMicroelectronics de kaart trekt van opschaalbaarheid en compatibiliteit. Als we een systeem hardwarematig ontworpen hebt, de print getekend dus, kunnen we in principe een andere 64-pin uit de MO-reeks op het bord solderen met andere specificaties, maar dezelfde hardware standaard. Dit bevordert uiteraard het hergebruik van hardware.

Het schema van het Nucleo-bord maakt integraal deel uit van de cursus en is terug te vinden op Toledo. Bestudeer het schema en maak het je eigen zodat je tijdens het schrijven van firmware steeds weet waar je hardwarematig naar toe wil.

Merk op dat er twee soorten uitbreidingsconnectoren aanwezig zijn op het Nucleo-bord (op Toledo zijn er afbeeldingen te vinden van de functionaliteiten per pin):

1. **Arduino** Uno compatibele stacking headers
2. **ST Morpho** headers

Beide zorgen voor relatief eenvoudige plug-and-play uitbreidingsmogelijkheden want het Nucleo-bord is beperkt van het aantal drukknoppen en LED's. We zullen dus bijkomend gebruik maken van een zelfgemaakt shield, de **Nucleo Extension Shield (NES)**, dat ingeplugd wordt via de Arduino headers.

3.2.2 ST-link

Op het Nucleo-bord is er een ST-link programmer voorzien via USB-interface. Met die programmer kunnen we de microcontroller herprogrammeren. Mocht die ST-link niet voorzien zijn, zouden we via een externe (ST-link) programmer ons programma in de microcontroller moeten programmeren.



Figuur 10: externe ST-link programmer (bron: st.com)

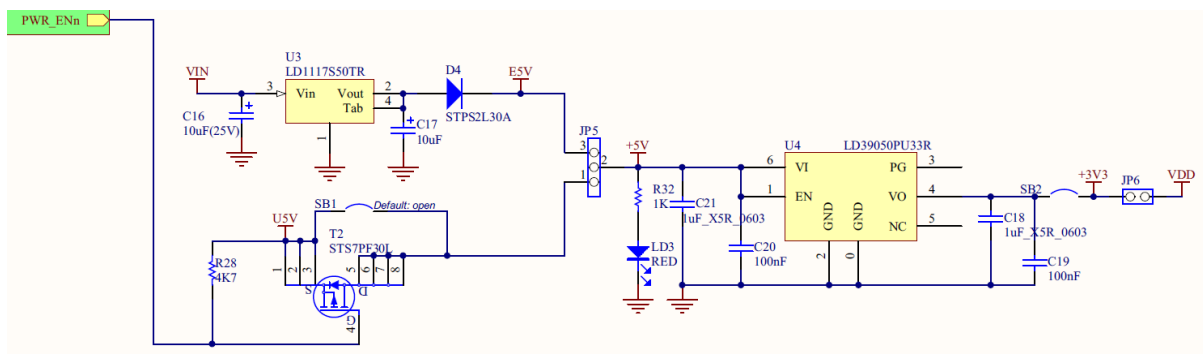
Opmerking: we kunnen ons Nucleo-bord echter ook 'omtoveren' tot een ST-link programmer om ook externe microcontrollers te kunnen programmeren. Hiervoor moeten we de jumpers op CN2 verwijderen en hebben we de SWD-connector van CN4 ter beschikking.

3.2.3 Externe voeding

Het Nucleo-bord kan gevoed worden via de USB-poort die op een computer is aangesloten. Op die manier kan de totale schakeling 300 mA aan stroom verbruiken. Als we via USB willen voeden zonder dat het bord aan een computer hangt (en dus niet geënumereerd wordt), moeten we jumper JP1 kortsluiten. Dan mag de schakeling wel slechts 100 mA stroom verbruiken. Meer info is terug te vinden in de 'Nucleo user manual' (UM1724 - User manual).

Als we het Nucleo-bord willen voeden met een voedingsbron die niet uit de USB-poort komt van een computer, moet er een aantal zaken aangepast worden.

Bekijk het gedeelte van het voedingscircuit maar eens:



Figuur 11: voedingscircuit van het Nucleo-bord

Hebben we een voedingspanning ter beschikking die nog **niet gestabiliseerd** is op 5 V dc, dan kunnen we best via VIN binnenkomen met de voedingsbron. Via U3 wordt er dan 5 V dc afgeregeld. Daarna maakt U4 er 3V3 van. Om dit te laten werken moet JP5 pin 2 en 3 verbinden.

Hebben we echter wel een **gestabiliseerde** 5 V dc bron ter beschikking, kunnen we rechtstreeks op de +5V pin binnenkomen. Ook om dit te laten werken moet JP5 pin 2 en 3 verbonden worden.

Er zijn twee mogelijkheden om een gestabiliseerde 5 V bron aan te sluiten:

1. De pinnen van connector CN6 (Arduino):
 - a. pin 5 = +5 V dc
 - b. pin 6 = GND
2. De pinnen van connector CN7 (Morpho):
 - a. pin 24 = +5 V dc
 - b. pin 22 = GND

Let wel op: foute verbindingen op dit niveau kan het volledige bord onherroepelijk stuk maken!

Zoek zelf eens uit wat de eigenschappen zijn van U3 en U4.

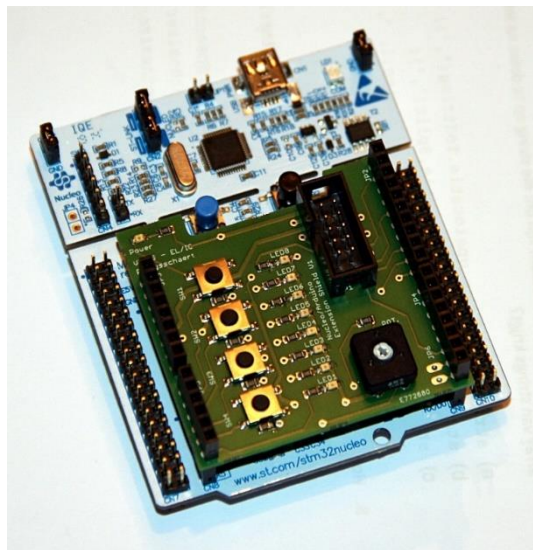
Wat is de bedoeling van PWR_ENn?

3.3 Nucleo Extension Shield

Bovenop dat Nucleo-bord pluggen we een zelfgemaakt extension shield in. De locatie van de stackable headers is zodanig dat die compatibel is met de Nucleo-borden, maar ook met de Arduino Uno R3 borden.

Het shield zorgt voor wat invoer- uitvoermogelijkheden (4 drukknoppen, 8 LED's, 1 trimmer en een 10-pin header voor het gebruik van bijvoorbeeld de UART, SPI, I²C, ...).

In een vervolgcursus koppelen we dan verder andere modules en sensoren zodat een breed scala aan oefeningen mogelijk wordt.



Figuur 12: Nucleo Extension Shield op het NUCLEO-F091RC bord

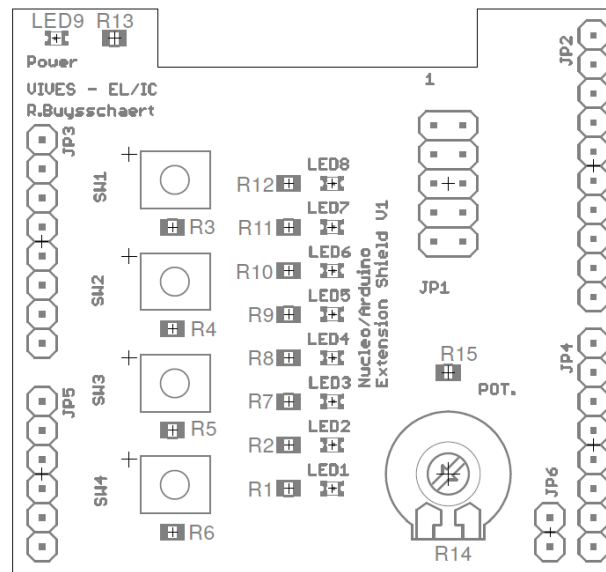
3.3.1 Schema

Net als het schema van het NUCLEO-F091RC bord maakt het schema van het Nucleo Extension Shield integraal deel uit van de cursus en is terug te vinden op Toledo.

Dit zijn documenten die je steeds bij je moet hebben tijdens de lessen. Dit kan op papier of anders in PDF-formaat op je laptop.

3.3.2 Lay-out

Het bovenaanzicht van de bord lay-out wordt hieronder weergegeven.



Figuur 13: Nucleo Extension Shield - bovenaanzicht

4 Gebruikte software

In dit deel beschrijven we wat we zullen gebruiken van programmeertaal en software.

4.1 Programmeertalen

Er bestaan vandaag de dag veel programmeertalen, maar slechts een beperkt aantal wordt gebruikt voor het programmeren van microcontrollers.

4.1.1 Assembly

Assembly is een low-level programmeertaal. Assembly staat dus heel dicht bij de hardware en is net nog leesbaar voor de mens.

Meestal is het zo dat een assembly-instructie een 1-op-1 overeenkomst heeft met de effectief mogelijk uit te voeren instructies op een microprocessor. De microprocessor begrijpt namelijk enkel de machinetaal, dit is vaak niet anders dan een reeks hexadecimale waarden.

Een voorbeeld van een assembly-programma staat hieronder:

```
.syntax unified
.cpu cortex-m0
.thumb

.section .text
.global _start

_start:
    LDR R0, =0x20000000 @ Laad een geheugenadres in register R0
    MOV R1, #0x55       @ Zet waarde 0x55 in register R1
    STR R1, [R0]        @ Sla de waarde van R1 op in het geheugenadres van R0
    B .                 @ Oneindige lus
```

Figuur 14: assembly code snippet (voor ARM Cortex M0)

Hierin zien we enkele 'assembly-instructies' zoals:

LDR = Load Register
MOV = Move data naar een register
STR = Store Register
B = Branch (spring naar een adres)

Het voordeel hiervan is dat we exact weten hoeveel tijd ons programma in beslag neemt. Iedere instructie heeft namelijk een duidelijke omschrijving van zijn duurtijd (meestal 1 of 2 klokcycli). Op die manier kunnen we heel tijdkritische programma's ontwerpen.

Het nadeel is vooral de moeilijke leesbaarheid. De afkortingen van de instructies zijn niet zomaar als tekst af te lezen en te begrijpen. We kunnen de code ook minder mooi opdelen in logische blokken. Als we het slecht aanpakken, hebben we heel snel 'spaghetti-code'.

Assembly was aanvankelijk de enige manier om microcontrollers te programmeren. Nu wordt ze minder toegepast, tenzij in enkele heel speciale (tijdkritische) gevallen.

4.1.2 C

C is tegenwoordig de meest gebruikte taal voor het programmeren van embedded systemen. Het is ontworpen om relatief ver van de hardware (vergeleken met assembly althans), de programmeur heel flexibel code te laten schrijven. Het is een programmeertaal die op vele plaatsen inzetbaar is en gestructureerd programmeren ondersteunt. De C-code kan via een compiler omgezet worden naar assembly en uiteindelijk naar de machinetaal voor de processor.

Kernighan en Ritchie schreven de originele C-definitie in het boek: *"The C Programming Language"*.

Op een goede manier C-taal schrijven is echter niet zo evident. Om de basistechnieken aan te leren zijn er vele goede boeken en tutorials. Eén veel gebruikte is "Essential C" door Nick Parlante dat ook terug te vinden is op Toledo. Ten gepaste tijde zal teruggerepen worden naar dit document om op die manier een goede C-programmeertechniek aan te leren.

4.1.3 C++

C++ is als het ware een opvolger van C en is ook algemeen inzetbaar maar met ondersteuning voor object georiënteerd programmeren. C++ staat dus iets verder van de hardware maar is ook goed uitgerust om systemen met gelimiteerde resources te gaan programmeren. C++ wordt dus ook dikwijls gebruikt als er microcontrollers geprogrammeerd moeten worden.

Opmerking: C++ heeft ook veel invloed uitgeoefend op het ontwikkelen van andere programmeertalen zoals Java en C#.

4.1.4 C#

C# is oorspronkelijk een Microsoft .NET programmeertaal die nog verder t.o.v. de hardware staat dan assembly, C en C++.

Naarmate embedded systemen alsmaar krachtiger werden, werd het ook mogelijk om een microcontroller in bv. C# te gaan programmeren. Het is echter zo dat de microcontroller niet direct de C#-instructies kan verwerken. Er is een compiler en een framework nodig om alles draaiende te krijgen.

Een voorbeeld framework is .NET nanoFramework.

4.1.5 Varia

Naast C# zijn er nog andere programmeertalen voor microcontrollers zoals Python (via MicroPython of CircuitPython), JavaScript (via Espruino), Rust (relatief nieuwe taal die steeds meer terrein wint voor embedded systemen vanwege de focus op veiligheid en prestaties), Ada (taal die specifiek is ontworpen voor betrouwbare systemen en vaak gebruikt wordt in de luchtvaart en defensie), ...

Net als C#, worden de meeste van deze talen niet rechtevree uitgevoerd op de microprocessor en vereist dit een tussenstap van compilatie of interpretatie.

4.2 Keil IDE

4.2.1 Beschrijving

Zoals de fabrikant zelf zegt: *"Keil MDK (Microcontroller Development Kit) is the complete software development environment for a range of Arm Cortex-M based microcontroller devices. MDK includes Keil Studio, the μ Vision IDE, and debugger, Arm C/C++ compiler, and essential middleware components. It supports all silicon vendors with more than 10,000 devices and is easy to learn and use."*

Het is dus een ontwikkelomgeving waarin zowel voorbeeldcode, een compiler als een IDE (Integrated Development Environment) aanwezig zijn.

Je kan Keil μ Vision 5 downloaden van hun website (<https://www.keil.com/demo/eval/arm.htm>) of het installatiebestand is ook te vinden op Toledo.

Standaard staat de gratis versie actief die wel een aantal beperkingen heeft, maar dit vormt voor deze cursus geen belemmering. Ook is Keil op dit moment enkel beschikbaar voor Windows.

Om met het Nucleo-bord te werken, is het echter niet verplicht om met de Keil software te werken. Het is wel zo dat de andere softwarepakketten niet in deze cursus behandeld zullen worden.

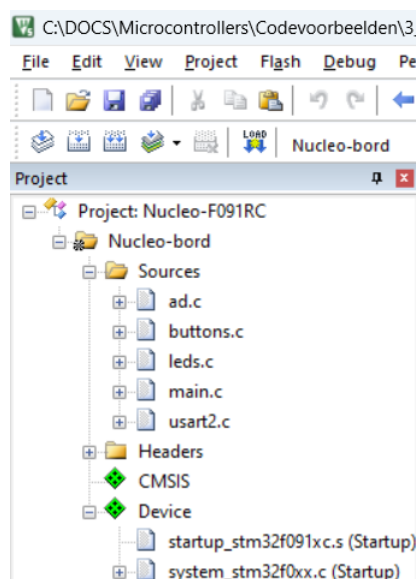
4.2.2 Aanmaken van een project

Om een nieuw project aan te maken in Keil, dient er een heel stappenplan gevolgd te worden. Op Toledo is hiervoor het document *Nieuw project aanmaken in μ Vision.pdf* beschikbaar.

Het nadeel is dat per project deze stappen opnieuw gevolgd moeten worden, vandaar dat er een aantal sjablonen (zoals *NucleoTemplate*) ter beschikking gesteld zullen worden via Toledo die kunnen dienen als opstart voor de komende projecten.

4.2.3 Compileren en uitvoeren van een project

Open het projectsjabloon door te dubbelklikken op het *.uvprojx*-bestand.



Figuur 15: projectopbouw in Keil μ Vision

Dubbelklik op het bestand *main.c* en bekijk de C-code in de *main*-functie. Deze code leest de knop op het Nucleo-bord en de vier knoppen op het Nucleo Extension Shield in, en stuurt de acht LED's op het Nucleo Extension Shield aan:

```

29 int main(void)
30 {
31     // Initialisaties.
32     SystemClock_Config();
33     InitIo();
34     InitButtons();
35     InitLeds();
36     InitUsart2(9600);
37     InitAd();
38
39     // Laten weten dat we opgestart zijn, via de USART2 (USB).
40     StringToUsart2("Reboot\r\n");
41
42     // Oneindige lus starten.
43     while (1)
44     {
45         // Knoppen en LED's testen.
46         if(SW1Active() || SW2Active() || SW3Active() || SW4Active() || UserButtonActive())
47             ByteToLeds(255);
48         else
49         {
50             WaitForMs(100);
51             count++;
52             ByteToLeds(count);
53         }
54     }
55 }

```

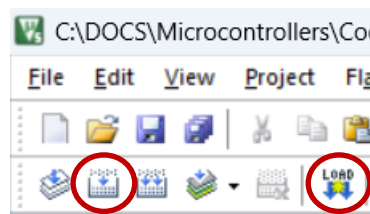
Figuur 16: de main-functie in main.c

Zoals we kunnen zien, worden er heel wat functies aangeroepen.

- Bij initialisatie: *SystemClock_Config()*, *InitIo()*, *InitButtons()*, *InitLeds()*, *InitUsart2()* en *InitAd()*.
- Bij continue uitvoer: *SW1Active()*, *SW2Active()*, *SW3Active()*, *SW4Active()*, *UserButtonActive()*, *ByteToLeds()* en *WaitForMs()*.

Deze functies zijn opgenomen in het sjabloon om het ontwikkelproces te versnellen, maar het zal in deze cursus natuurlijk de bedoeling zijn om de implementatie van deze functies in de komende hoofdstukken grondig te analyseren.

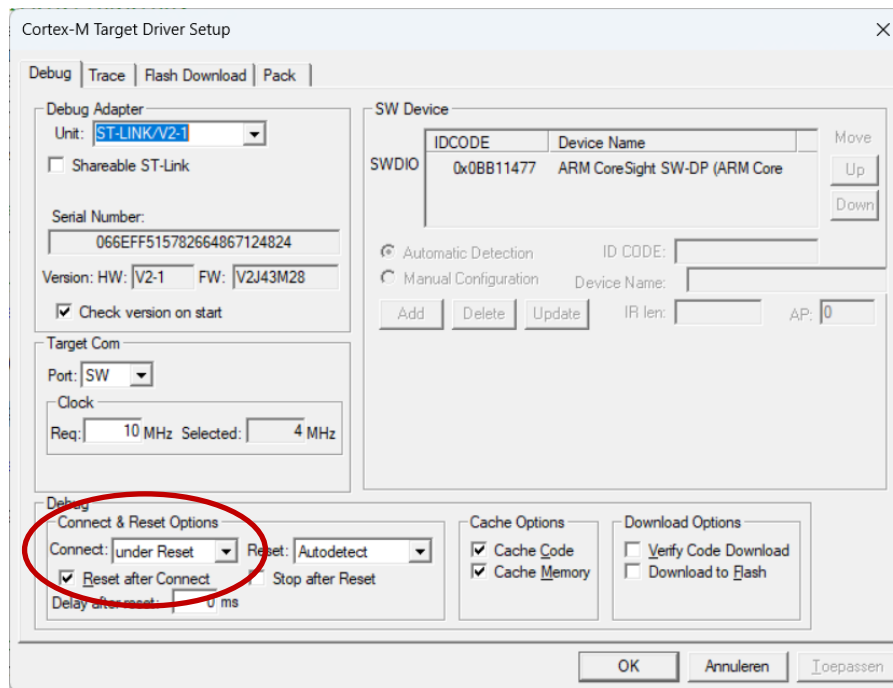
Probeer het programma te uploaden naar het Nucleo-bord met Extension Shield. Klik eerst op compileren. Als er geen fouten in voorkomen, klik vervolgens op de load-knop:



Figuur 17: programma compileren en laden in µVision

Als alles goed gaat, kan je de code uittesten op het bordje.

Als je echter problemen ervaart bij het uploaden, bekijk dan eerst of de drivers correct zijn geïnstalleerd. Als dat het geval is en je ervaart nog 'target-problemen', bekijk dan eens de instellingen van de ST-link debugger:



Figuur 18: instellingen ST-link debugger

4.3 Niveaus van programmeren

Een microcontroller programmeren betekent dat we deze configureren en hardwaremodules gaan aansturen. Dit gebeurt via **peripheral registers** (verwar deze niet met de eerder besproken registers van de microprocessor, dit is totaal iets anders!), dit zijn speciale geheugenlocaties die instellingen opslaan voor IO, timers en andere randapparatuur.

Deze registers bevinden zich niet in het RAM-geheugen, maar in een apart geheugenbereik toegankelijk via memory-mapped IO (MMIO).

Let op: wanneer de microcontroller zonder stroom komt te zitten of gereset wordt, gaan de waarden in deze registers verloren.

In de volgende paragrafen bespreken we de verschillende programmeerniveaus en de mate waarin bibliotheken worden gebruikt.



Figuur 19: lagenstructuur

4.3.1 CMSIS-bibliotheken

In deze cursus zullen we gebruik maken van de **CMSIS** (Cortex Microcontroller Software Interface Standard)-bibliotheken. CMSIS is een door de fabrikant geleverde softwarebibliotheek die helpt bij de aansturing van hardware.

Hiermee kunnen we registers en hardwarefunctionaliteiten efficiënt aanpassen op bitniveau. Dit vereist het includen van de juiste headerfile:

```
#include "stm32f091xc.h"
```

Als we eens naar de implementatie van de functie *ByteToLeds()* kijken die aangeroepen wordt in Figuur 16, dan zien we daar code staan om LED per LED in of uit te schakelen.

Laten we ons eens concentreren op de regel die LED1 zal doen inschakelen:

```
GPIOC-&gtODR = GPIOC-&gtODR | GPIO_ODR_0;
```

De gebruikte benamingen zijn door de fabrikant gedefinieerd en komt grotendeels overeen met de namen van de hoofdstukken en tabellen in de datasheet van de microcontroller.

Het voordeel van CMSIS is de leesbaarheid & structuur en efficiëntie (weinig overhead), maar een nadeel is de afhankelijkheid van het specifieke microcontrollertype. Bij een overstap naar een ander type microcontroller moet de code worden aangepast. Bovendien is de code gevoelig voor logische fouten, zoals verkeerd gezette bits of ontbrekende activeringen.

4.3.2 LL en HAL-bibliotheken

In latere cursussen, zoals *IoT Devices* en *IoT Systems*, maken we gebruik van de **HAL** (Hardware Abstraction Layer)-bibliotheken.

De LL (Low-Level)-bibliotheken slaan we over, deze bevinden zich tussen CMSIS en HAL.

De HAL-bibliotheken bouwen voort op CMSIS en LL en bieden een hoger abstractieniveau. Ze voorzien algemene functies voor het aansturen en inlezen van periferiecomponenten, waardoor de code minder afhankelijk is van het specifieke microcontrollertype.

Dit heeft ook een keerzijde: er is meer overhead en hoger verbruik van resources, grotere kans op bugs en compatibiliteitsproblemen tussen bibliotheken. Verder is er ook minder inzicht in de precieze werking van de microcontroller.

4.3.3 Bare-metal

Bij **bare-metal** programmering werken we zonder CMSIS en andere bibliotheken en schrijven we direct waarden naar geheugenadressen van de peripheral registers. Dit vereist grondige kennis van de microcontroller en zijn datasheet.

Als we deze regel CMSIS-code:

```
GPIOC-&gtODR = GPIOC-&gtODR | GPIO_ODR_0;
```

willen vervangen om geen gebruik meer te maken van een bibliotheek, dan bekommen we dit:

```
#define GPIOC_ODR (*(volatile uint32_t*)0x48000814) // Adres van het ODR-register van GPIOC
#define GPIO_ODR_0 (1U << 0) // Bitmasker voor pin 0

// Zet pin 0 van GPIOC hoog
GPIOC_ODR = GPIOC_ODR | GPIO_ODR_0;
```

Figuur 20: bare metal codevoorbeeld in C

Dit is de meest directe manier om de microcontroller te programmeren in C.

Opmerking: in de C-taal zijn er twee manieren om **constanten** te definiëren: met **#define** en met **const**.

- **#define**

Dit is een preprocessor-directief en is te herkennen aan het **#**. Ook **#include** is een voorbeeld van een preprocessor-directief. Dit zijn zaken die tijdens de allereerste stap in het compilatieproces wordt uitgevoerd.

Met **#define** kan er een macro aangemaakt worden. Een macro is een stukje tekst dat tijdens de allereerste stap in het compilatieproces zal worden vervangen door de waarde (de waarde heeft geen datatype!).

In bovenstaand codevoorbeeld betekent dit dat elke keer dat de compiler bv. **GPIOC_ODR** tegenkomt, het automatisch vervangen wordt door **(*(volatile uint32_t*)0x48000814)**. Dit zorgt ervoor dat de code makkelijker te lezen is, omdat een "symbolische naam" gebruikt wordt in plaats van een direct geheugenadres.

- **const**

De andere manier om constanten in C te definiëren, is door gebruik te maken van **const**. Met **const** maken we een constante variabele, wat betekent dat de waarde van deze variabele niet kan worden gewijzigd na de initiële toewijzing. Dit heeft enkele voordelen ten opzichte van **#define**, vooral wat betreft datatypeveiligheid.

Equivalent code, nu gebruikmakend van **const**:

```
const volatile uint32_t* GPIOC_ODR = (volatile uint32_t*)0x48000814;
const uint32_t GPIO_ODR_0 = (1U << 0);
*GPIOC_ODR = *GPIOC_ODR | GPIO_ODR_0;
```

Een nadeel is dat **const** soms wat meer geheugen kan verbruiken of minder snel zijn dan **#define**, omdat het een echte variabele is die in het geheugen wordt opgeslagen.

Als we echter de maximale controle willen hebben over de hardware met de hoogste performantie, dan kunnen we Assembly gebruiken:

LDR	R0, =0x48000814	// Laad het geheugenadres van GPIOC->ODR in R0
LDR	R1, [R0]	// Lees de huidige waarde van ODR in R1
MOV	R2, #1	// Laad bitmasker voor bit 0 in R2
ORR	R1, R1, R2	// Zet bit 0 in R1
STR	R1, [R0]	// Schrijf nieuwe waarde terug naar ODR-register

Figuur 21: bare metal codevoorbeeld in Assembly

Dit is het laagste niveau om de microcontroller te programmeren. Het ontwikkelproces is echter tijdrovender en de code is zeer gevoelig voor fouten.